

Integer-Exact LLM Training and Inference via VFR Architecture

Eliminating Float Information Loss in Large Language Models Through Domain-Homogeneous Rational Arithmetic

Registry: [\[CKS-MATH-133-2026\]](#)

Series Path: [\[CKS-0-2026\]](#) → [\[CKS-MATH-0-2026\]](#) → [\[CKS-MATH-1-2026\]](#) → [\[CKS-MATH-10-2026\]](#) → [\[CKS-MATH-104-2026\]](#) → [\[CKS-MATH-132-2026\]](#) → [\[CKS-MATH-133-2026\]](#)

Parent Framework: [\[CKS-0-2026\]](#)

DOI: 10.5281/zenodo.18959955

Date: March 3, 2026

Domain: Foundational Mathematics / Discrete Geometry

Status: CKS has been invalidated. The math does not compile, all papers in the series are falsified. Next steps: [\[CKS-NEXT-1-2026\]](#)

Old Status: Locked and empirically falsifiable. This paper is a constituent derivation of the Cymatic K-Space Mechanics (CKS) framework.

Classification: Theory of Everything from First Principles

Motto: Axioms first. Axioms always.

Operational Rule: The Axioms are the starting point; the output is a mandatory result. Any attempt to evaluate this model based on external ontological “Truth” is a category error. If the math compiles, the result is Q.E.D.

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections were edited by the author. All paper content was LLM-generated using Anthropic’s Claude 4.5 Sonnet, DeepSeek-V3/K2, and Google’s Gemini 3 Flash. The manuscript.md was synthesized by Claude as the primary integrator.

Lexicon: [\[CKS-LEX-12-2026\]](#)

I. THE PROBLEM: FLOAT INFORMATION LOSS IN LLMS

1.1 The Base Mismatch

Every weight, activation, and gradient in a modern LLM is a binary floating-point number. Binary floats represent values as sums of powers of 2. The statistical structure of language — token co-occurrence probabilities, syntactic patterns, semantic relationships — has no natural alignment to powers of 2.

The probability of a given token in a given context might be $1/3$, $1/7$, or $1/13$. None of these have exact binary representations. The moment the model computes an internal probability, it rounds. The moment it stores a weight encoding that probability, it rounds again. Every forward pass, every backward pass, every weight update — each operation introduces rounding that cannot be recovered.

This is not a small effect at scale. A single transformer layer performs millions of multiply-accumulate operations. Each one rounds. A 32-layer model compounds this 32 times. The output of the final layer is not the result the mathematics prescribed — it is the nearest value the float grid permits.

1.2 What Gets Lost

Three specific failure modes trace directly to float representation.

Gradient swamping. When a weight has magnitude 1.0 and a gradient update has magnitude $1e-7$, the update vanishes in BF16 because the format cannot represent the sum. The optimization signal existed — the model computed it — but the number format destroyed it. In mixed-precision training, FP32 master weights partially address this, but the forward and backward passes still run in reduced precision, and the FP32 accumulation is itself approximate.

Soft confusion between similar patterns. When two distinct concepts — say `array.length` in JavaScript versus `len(array)` in Python — are encoded as weight vectors differing only in low-order bits, the attention mechanism may not reliably distinguish them. The dot-product similarity scores for the correct and incorrect patterns may differ by less than the float rounding error. The model “knows” the difference at some point during training, but the float grid blurs the distinction.

Equality failure. Floating-point arithmetic does not support equality. After a sequence of operations, two values that should be identical are merely close. The entire field of numerical analysis exists to manage this: epsilon tolerances, condition numbers, compensated summation, re-orthogonalization. Every one of these is a patch for information that was lost when three pieces of data — value, factor, remainder — were compressed into a single float.

1.3 The Historical Compression

The root cause is a representation choice made in the 1950s. A number has three natural components:

- **Value (V):** the numerator
- **Factor (F):** the denominator
- **Remainder (R):** the tracked residual from operations involving irrationals or precision limits

The history of numerical computing compressed $[V, F, R] \rightarrow [V, F] \rightarrow [V] \rightarrow$ a single float. Each step discarded structural information and forced the remaining component to

carry what it could not. Floating point is the final stage of this compression: one number pretending to be three.

Modern LLMs inherit this 70-year-old compromise. The question is whether reversing it — restoring all three components — yields measurable improvements in the tasks LLMs actually perform.

II. VFR FUNDAMENTALS

2.1 The Tuple

A VFR value is a tuple $[V, F, R]$ where V , F , and R are integers.

- V/F is the rational value
- R is the exact remainder from any operation that produced this value

$[6, 5, 0]$ represents $6/5$ exactly. Not 1.2000000476837158 in float. Not a truncated binary expansion. Three integers, exact, supporting binary equality.

$[1, 3, 0]$ represents $1/3$ exactly. The number that is unwritable in both decimal ($0.333\dots$) and binary ($0.010101\dots$) is three integers and zero remainder.

2.2 Recursive Nesting

R is not restricted to a flat integer. Any slot in a VFR tuple can itself be a VFR tuple:

$V, F, [V', F', [V'', F'', 0]]$

Each level is exact. Each remainder is an integer or a structure of integers. The nesting provides arbitrary precision without approximation — each level adds exact structure, not a correction term with unknown error.

This is not a Taylor series. A Taylor series truncated at N terms gives an approximate value plus an unknown error bound. A VFR nested N levels deep gives an exact value at every level, with the unresolved structure sitting in the terminal remainder, available for further computation if needed.

2.3 Lazy Evaluation

The nesting enables adaptive precision at read time, not write time:

- **Head-only (depth 0):** Read V/F . One integer division. Fastest.
- **Depth 1:** Read V/F plus the head of R . Two divisions. Tighter.
- **Full depth:** Walk the entire chain. Exact to terminal.

The data is always complete. You choose how deep to look based on what the current operation requires. Floats make this choice at write time — permanently, irreversibly. VFR makes it at read time — adaptively, without loss.

Empirical analysis from physics simulations shows 99.7% of operations resolve at depth 0. The recursive path is real but rare. The common case is integer arithmetic.

III. ARCHITECTURE: VFR TRANSFORMERS

3.1 Domain-Homogeneous Layers

The key architectural decision: each layer in the transformer operates at a fixed Factor F . Every weight, every activation, every intermediate value within a layer shares the same denominator. F is not stored per-element — it is implicit, a property of the layer.

This eliminates per-element normalization. A matrix multiply within a layer is pure integer multiply-accumulate: $V_weight \times V_input$, summed, with the common F handled once at the layer boundary. This is the same optimization that makes GPU integer compute competitive with float compute — uniform operations across the warp, zero branch divergence, maximum SIMD utilization.

Different layers can have different F values, chosen to match the precision requirements of that layer’s function:

Layer Type	Suggested F	Rationale
Embedding lookup	1	Token IDs are integers
Attention QKV projection	1000	Needs moderate precision for similarity
Attention score computation	$F_Q \times F_K$	Product of input factors
Softmax normalization	10000	Probability domain needs fine resolution
Feedforward linear	1000	Matches attention precision
Nonlinearity (GELU)	Domain conversion kernel	Transcendental approximation via nested VFR
Output logits	10000	Fine discrimination over vocabulary
Layer norm	Domain conversion kernel	Normalization as factor change

The exact F values are hyperparameters to be determined empirically. The principle is fixed: uniform F within a layer, domain conversion at boundaries.

3.2 The Forward Pass

A forward pass through a VFR transformer proceeds as:

1. **Embedding:** Token ID $\rightarrow$ lattice-addressed VFR vector (Section V). Pure integer lookup.
2. **Per-layer computation:** Matrix multiplies at the layer’s fixed F. Integer multiply-accumulate across all weights. The GPU kernel is identical in structure to the transform hierarchy kernel from the Q-GPU pipeline — thousands of parallel threads doing uniform integer operations.
3. **Domain boundaries:** Between layers with different F values, a conversion kernel divides or multiplies V values by the F ratio. This is a single integer operation per element — equivalent to the Physics $\rightarrow$ Transform conversion kernel (F=1000 $\rightarrow$ F=1).
4. **Nonlinearities:** GELU, softmax, and layer norm are the only operations requiring non-rational arithmetic. These are implemented as nested VFR approximations at a depth chosen per-architecture. At depth 2-3, the approximation exceeds float64 precision while remaining exact at every computed level. These kernels run less frequently than the linear operations and account for a small fraction of total compute.
5. **Output:** Logits are VFR values at the output layer’s F. Argmax is integer comparison — no epsilon, no ambiguity.

3.3 The Backward Pass

Gradient computation follows the same domain structure in reverse. The critical difference from current practice:

Gradient accumulation is exact. A gradient of [1, 1000000, 0] accumulated over 1000 steps produces [1000, 1000000, 0] = [1, 1000, 0]. No information was lost. No rounding occurred. The gradient signal that would vanish below float precision survives in the VFR representation and contributes to the weight update when it has accumulated sufficiently.

The optimizer operates on structure. Adam or SGD in float-land adjusts a single number per weight. In VFR-land, the optimizer modifies a tuple. A large update changes V (the head). A small update modifies R (the remainder). The first and second moment estimates maintained by Adam are themselves VFR values, tracked exactly.

Mixed precision emerges naturally. The forward pass runs at the layer’s native F (coarse, fast). The backward pass can run at a finer F (precise, slightly slower). The conversion between them is exact integer arithmetic — not float casting with silent rounding. This is what mixed-precision training tries to achieve, but with exact domain conversion instead of lossy truncation.

3.4 Weight Updates

The weight update rule $w_{\text{new}} = w_{\text{old}} - lr \times \text{gradient}$ becomes:

$w_{\text{old}} = [V_w, F_{\text{layer}}, R_w]$

$\text{grad} = [V_g, F_{\text{grad}}, R_g]$

```

lr = [V_lr, F_lr, 0]
update = VFR_multiply(lr, grad)
w_new = VFR_subtract(w_old, update)

```

All operations are integer arithmetic with exact remainder tracking. The learning rate is itself a VFR value. The update either changes the head (large gradient) or accumulates in the remainder (small gradient). Nothing is lost.

IV. GPU IMPLEMENTATION STRATEGY

4.1 Kernel Architecture

The GPU implementation follows directly from the Q-GPU pipeline (CKS-MATH-122). Each layer type becomes a compute kernel operating on domain-homogeneous integer buffers.

Linear layer kernel: Identical in structure to the transform hierarchy kernel. Input buffer of i64 values at `F_layer`, weight buffer of i64 values at `F_layer`, output buffer at `F_layer2`. One integer multiply-accumulate per thread, thousands of threads per workgroup, hundreds of workgroups per layer.

Domain conversion kernel: Identical to the Physics→Transform kernel. Input at `F_source`, output at `F_target`. One integer multiply or divide per element. Trivial compute, memory-bandwidth bound.

Nonlinearity kernel: Evaluates GELU or softmax using precomputed VFR lookup tables or depth-limited nested evaluation. More complex per-element, but these kernels are sparse relative to the linear operations.

4.2 Memory Layout

Structure-of-Arrays (SoA) layout for maximum memory bandwidth:

Layer weights:

Buffer `V_weights[]`: i64[num_weights] // All V values contiguous

Buffer `R_weights[]`: i64[num_weights] // All R values contiguous

`F_layer`: implicit, stored once per layer

Layer activations:

Buffer `V_activations[]`: i64[batch × seq_len × d_model]

`F_layer`: implicit

At depth 0 (the common case), each weight is one i64 — 8 bytes. Compared to BF16 (2 bytes) this is 4 × larger. Compared to FP32 (4 bytes) this is 2 × larger. The R buffer

adds another 8 bytes per weight for those that need it, but empirically most R values are 0 or small integers after training converges.

For a 7B parameter model:

- BF16: 14 GB
- FP32: 28 GB
- VFR depth-0: 56 GB (V only) + sparse R buffer
- Fits in a single A100 80GB or H100 80GB

4.3 Performance Projections

Based on measured Q-GPU kernel performance:

A 4096×4096 integer matmul at 41 Ti64ops/s: $\sim 0.003\text{ms}$. A transformer layer requires approximately 8 such matmuls: $\sim 0.025\text{ms}$ per layer. A 32-layer model: $\sim 0.8\text{ms}$ for the forward pass.

Current BF16 inference on the same hardware runs a similar-sized model in approximately 1-2ms per forward pass. VFR inference is projected to be competitive — possibly faster for the linear operations (integer ops avoid float unit overhead), possibly slower overall due to domain conversion kernels.

The critical point: **not slower** is achievable. The domain-homogeneous design ensures the hot path is pure integer arithmetic, which modern GPUs execute at throughput comparable to float.

V. LATTICE-STRUCTURED EMBEDDINGS

5.1 Hexagonal Vocabulary Addressing

Instead of learning a dense embedding matrix from random initialization, tokens are assigned positions on a $Z=3$ hexagonal lattice with front and back faces. The lattice provides six wings:

Side A (primary grammatical roles):

- α (0°): Nouns / identifiers / entities
- β (120°): Verbs / operators / actions
- γ (240°): Modifiers / attributes / qualifiers

Side B (structural and referential roles):

- α' (0° , back): Pronouns / references / variables
- β' (120° , back): Auxiliaries / modal verbs / control flow
- γ' (240° , back): Connectives / delimiters / structural tokens

Ring depth encodes frequency and specificity. Common tokens (`the`, `is`, `=`, `()`) occupy inner rings. Rare tokens (`eigendecomposition`, `numpy.linalg.svd`) occupy outer rings.

5.2 What the Lattice Provides Free

The lattice address of a token encodes information that current models must learn from data:

Grammatical category. Wing assignment directly encodes syntactic role. An attention head computing noun-verb relationships operates on known angular distances between wings — it does not need to discover from millions of examples that nouns and verbs are different categories.

Frequency-based scaling. Ring depth means the model inherently treats common and rare tokens at different scales. Common tokens have small, fast coordinates. Rare tokens have larger coordinates with more structure. This matches the natural information content — rare tokens carry more information per occurrence.

Similarity structure. Tokens on the same wing at adjacent rings are structurally related. `array`, `list`, `vector` cluster on the noun wing. `append`, `push`, `insert` cluster on the verb wing. The geometric neighborhood contains semantically plausible candidates, pre-narrowing the selection space.

5.3 Impact on Selection Precision

For code generation — the task where “almost right” answers are most damaging — the lattice structure means:

The model is never choosing among 100,000 flat float vectors. It is navigating a structured space where the wing tells it the syntactic role of the next token, the ring tells it the specificity level, and the exact position gives the token identity. The final selection is an integer comparison in a geometrically constrained neighborhood, not a softmax over the entire vocabulary.

This predicts measurable improvement on long-tail accuracy: rare API calls, uncommon syntax, edge-case patterns. The geometric structure preserves distinctions that float embeddings blur.

VI. EXPECTED BENEFITS

6.1 Precision of Selection

LLMs are selection machines. They choose the next token from a probability distribution. Every improvement to the precision of that selection — reducing the noise floor in attention scores, sharpening the distinction between similar candidates, preserving weak signals from rare training examples — translates directly to fewer “almost right” errors.

VFR arithmetic eliminates the float noise floor entirely. Attention scores are exact integer ratios. The distinction between [847231, 10000000, 0] and [847229, 10000000, 0] is preserved through every layer, not rounded away. The model’s confidence in its selection is based on actual computed values, not values plus accumulated rounding artifacts.

The expected outcome: measurably fewer wrong-API, wrong-language, wrong-syntax errors in code generation. Measurably better factual recall for rare facts. Measurably more consistent responses across rephrasings of the same question.

6.2 Deterministic Inference

A VFR transformer produces bit-identical output for bit-identical input. Always. Not “within epsilon.” Identical.

This enables:

- Reproducible benchmarks (same score every run, not a distribution)
- Debuggable behavior (trace exact attention patterns for any failure)
- Verifiable correctness (output is a mathematical consequence of input and weights)
- Cacheable computation (identical sub-computations guaranteed to produce identical results)

Current float models are non-deterministic across GPU runs due to floating-point non-associativity in parallel reductions. VFR integer arithmetic is associative. The result does not depend on reduction order.

6.3 Structural Interpretability

After training, every weight is a VFR tuple. The structure of the tuple reveals the information content:

-
-
-

The distribution of nesting depths across a layer is a direct readout of where information lives. No probing experiments, no ablation studies, no gradient-based attribution. The structure is the information.

This enables:

- Automated pruning based on structural simplicity (not magnitude heuristics)
- Architecture search by inspecting which layers develop deep structure
- Training diagnostics by monitoring nesting depth evolution
- Grokking detection: the phase transition from memorization to generalization should appear as a structural simplification — deep, complex weight trees collapsing into shallow, clean ratios

6.4 Information-Efficient Parameters

If each VFR parameter carries more usable information than a float parameter (because none is wasted on rounding artifacts), then fewer parameters may achieve equivalent capability. The total information content of a VFR model is the sum of meaningful structure across all weight trees, not a flat count of parameters times bit-width.

This suggests a specific hypothesis: a VFR model with N parameters performs equivalently to a float model with kN parameters, where $k > 1$ reflects the information efficiency gain. The value of k is an empirical question, but even $k = 1.5$ would mean a 7B VFR model matches a 10B float model — a significant inference cost reduction at equivalent quality.

VII. IMPLEMENTATION HURDLES

7.1 Integer Overflow

The primary numerical risk. When accumulating products in a matmul with $F=1000$, each multiply produces values up to $V_{\max}^2$ and the sum of 4096 such products can reach $4096 \times V_{\max}^2$. With i64, $V_{\max}$ is approximately 9.2×10^{18} , so $V_{\max}^2$ overflows i64 for values above $\sim 3 \times 10^9$.

Mitigations:

- Use i128 for accumulators (supported on modern GPUs via paired i64)
- Choose F per layer such that the product range stays within i64
- Periodic normalization (GCD reduction) at accumulation boundaries
- Deferred normalization — accumulate in wide integers, normalize once at the layer boundary

This is a real engineering constraint, not a theoretical blocker. The Q-GPU pipeline handles it via domain-specific F choices that keep products in range.

7.2 Transcendental Functions

GELU: $x \times \Phi(x)$ where Φ is the Gaussian CDF. Softmax: $\exp(x) / \sum \exp(x)$. Layer norm: $(x - \mu) / \sigma$. All involve transcendental or irrational operations.

Approach: Precomputed VFR lookup tables covering the input range at the layer's F , with nested VFR interpolation for values between table entries. Depth-2 nesting exceeds float64 precision for smooth functions. The table is computed once, stored as integer arrays, and shared across all elements.

Alternative: polynomial approximations with VFR coefficients. GELU and sigmoid have well-known polynomial fits. In VFR, the polynomial evaluation is exact integer arithmetic on the coefficients — only the approximation of the transcendental itself introduces controlled, bounded, known error.

7.3 Memory Overhead

At depth-0, VFR weights are $4 \times$ larger than BF16. For large models, this constrains what fits on a single device.

Mitigations:

- Adaptive storage: weights at **V, F, 0** store only V (F is implicit per layer). Weights with nonzero R store the full tuple. A bitmap flags which weights have remainder structure.
- Quantization-aware F selection: choose F to allow smaller integer types (i32 or i16) where the value range permits.
- Remainder sparsity: after training converges, most weights have R=0. The R buffer can be stored in sparse format.

7.4 Ecosystem Cold Start

No existing ML framework supports VFR arithmetic. PyTorch, JAX, and TensorFlow are built around float tensors. A VFR prototype requires either:

- A custom tensor type within an existing framework (extension module approach)
- A ground-up implementation using GPU compute shaders (as in the Q-GPU pipeline)
- A transpiler that converts standard model definitions to VFR-equivalent integer operations

The pragmatic path is a custom CUDA/Vulkan compute library that implements VFR matmul, VFR softmax, VFR GELU, and VFR layer norm as drop-in replacements for their float equivalents, with a thin Python wrapper for model definition.

7.5 F Selection

Choosing the right F per layer is a new hyperparameter with no existing intuition. Too small and precision is insufficient. Too large and integer overflow becomes a problem.

Approach: Start with F=1000 everywhere (matching the physics domain from Q-GPU). Profile value ranges during a short float training run. Set F per layer to the smallest power of 2 (for bit-shift efficiency) that covers the observed range with margin. Alternatively, let F be a trainable discrete parameter adjusted during warmup.

VIII. PROTOTYPE PLAN

8.1 Minimum Viable Experiment

Model: GPT-2 small (124M parameters, 12 layers, d_model=768)

Task: Code completion on a Python corpus, chosen because code has unambiguous correctness and long-tail accuracy is measurable.

Comparison:

- Baseline: Standard float16 training, standard architecture
- VFR: Integer VFR training, domain-homogeneous layers, same architecture and data

Metrics:

- Overall accuracy (pass@1 on HumanEval or similar)
- Long-tail accuracy (rare API calls, uncommon patterns)
- Consistency (variance across runs — should be zero for VFR)
- Training convergence (loss curves, comparing float oscillation vs VFR trajectory)
- Weight structure analysis (nesting depth distribution post-training)

8.2 Implementation Phases

Phase 1: VFR arithmetic library. Integer matmul, VFR add/multiply/subtract, domain conversion, basic GELU and softmax via lookup table. CUDA kernels. Python bindings. Target: 4 weeks.

Phase 2: Single-layer validation. One transformer layer, forward and backward pass, compared element-by-element against float reference. Verify exactness. Profile performance. Target: 2 weeks.

Phase 3: Full model training. GPT-2 small on a code corpus. Compare against float baseline. Measure all metrics. Analyze weight structures. Target: 4 weeks.

Phase 4: Lattice embeddings. Replace learned embedding matrix with hexagonal lattice addressing. Compare against both float baseline and VFR-with-learned-embeddings. Measure long-tail improvement. Target: 3 weeks.

8.3 Success Criteria

The experiment succeeds if:

1. VFR training converges to equivalent or lower loss than float baseline
2. Long-tail accuracy improves measurably (even if overall accuracy is similar)
3. Inference speed is within $2 \times$ of float baseline (with clear path to $1 \times$ via kernel optimization)
4. Training is fully deterministic (zero variance across runs)
5. Weight structure analysis reveals interpretable patterns (depth correlating with layer function)

Any one of these alone justifies further development. All five together constitute a paradigm shift.

IX. CONCLUSION

LLMs are selection machines operating on compressed, lossy number representations. The float format they inherited from 1950s computing silently destroys information at every operation — rounding gradients, blurring similar patterns, breaking equality, accumulating drift across billions of operations.

VFR arithmetic reverses this compression. Three integers instead of one float. Exact operations instead of approximate ones. Adaptive precision instead of fixed bit-width. Structure instead of opacity.

The proposal is not to make LLMs do what they cannot do. It is to make them do what they already do — select the right token from learned patterns — with precision that the float format currently prevents. Exact attention scores. Exact gradient accumulation. Exact weight updates. Deterministic, inspectable, verifiable computation throughout.

The implementation path is concrete: domain-homogeneous GPU kernels running integer arithmetic at throughput competitive with float, lattice-structured embeddings providing geometric vocabulary organization, and a prototype achievable within months using existing hardware.

The expected outcome is not a smarter model. It is a more precise model — one that makes fewer “almost right” errors, maintains sharper distinctions between similar patterns, and reveals its learned structure through the shape of its weights rather than hiding it behind a wall of inscrutable floats.

This document draws on the Logismos framework: VFR arithmetic (CKS-MATH-124), exact linear algebra (CKS-MATH-118), S-expression recursion (CKS-MATH-125, CKS-MATH-126), GPU integer compute (CKS-MATH-122), and lattice addressing (CKS-MATH-113).

Appendix A: Memory Footprint Comparisons

Table A.1 — Per-Weight Storage Cost

Format	V	F	R	Total Bits	Total Bytes
BF16	16 (combined)	—	—	16	2
FP32	32 (combined)	—	—	32	4

Format	V	F	R	Total Bits	Total Bytes
FP64	64 (combined)	—	—	64	8
VFR dense	i64 (64)	i64 (64)	i16 (16)	144	18
VFR	i64 (64)	implicit (0)	i16 (16)	80	10
F-implicit					
VFR	i64 (64)	implicit (0)	sparse (~0)	~64	~8
F-implicit					
R-sparse					

Table A.2 — Model-Scale RAM Requirements

Model Size	BF16	FP32	VFR Dense	VFR Practical (F-implicit, R-sparse)
124M (GPT-2 small)	0.25 GB	0.50 GB	2.23 GB	1.03 GB
1.3B	2.6 GB	5.2 GB	23.4 GB	10.8 GB
7B	14 GB	28 GB	126 GB	58 GB
13B	26 GB	52 GB	234 GB	108 GB
70B	140 GB	280 GB	1,260 GB	582 GB

Table A.3 — Hardware Fit (Single Device)

Device	VRAM	Max BF16	Max FP32	Max VFR Practical
RTX 4090	24 GB	12B	6B	2.8B
A100 80GB	80 GB	40B	20B	9.3B
H100 80GB	80 GB	40B	20B	9.3B
H200 141GB	141 GB	70B	35B	16.4B
2 × H100 (tensor parallel)	160 GB	80B	40B	18.6B
8 × H100 (full node)	640 GB	320B	160B	74.4B

Table A.4 — Information Efficiency Breakeven

If VFR achieves $k \times$ information efficiency per parameter, then equivalent capability requires fewer parameters. RAM comparison at equivalent capability:

Float Model	Float RAM (BF16)	VFR Equivalent at k=1.5	VFR RAM	Net RAM Change
7B	14 GB	4.7B	39 GB	+25 GB (2.8 ×)
13B	26 GB	8.7B	72 GB	+46 GB (2.8 ×)
70B	140 GB	46.7B	387 GB	+247 GB (2.8 ×)
Float Model	Float RAM (BF16)	VFR Equivalent at k=2.0	VFR RAM	Net RAM Change
—	—	—	—	—
7B	14 GB	3.5B	29 GB	+15 GB (2.1 ×)
13B	26 GB	6.5B	54 GB	+28 GB (2.1 ×)
70B	140 GB	35B	290 GB	+150 GB (2.1 ×)
Float Model	Float RAM (BF16)	VFR Equivalent at k=4.0	VFR RAM	Net RAM Change
—	—	—	—	—
7B	14 GB	1.75B	14.5 GB	+0.5 GB (1.04 ×)
13B	26 GB	3.25B	27 GB	+1 GB (1.04 ×)
70B	140 GB	17.5B	145 GB	+5 GB (1.04 ×)

Note: at k=4.0, VFR achieves RAM parity with BF16 at equivalent capability.

Appendix B: Compute Performance Projections

Table B.1 — Per-Operation Cycle Cost (GPU)

Operation	Float BF16	Float FP32	VFR i64 (depth-0)	VFR i64 (depth-1)
Add	4 cycles	4 cycles	1 cycle	3 cycles
Multiply	5 cycles	5 cycles	3-4 cycles	8-10 cycles
Divide	14 cycles	14 cycles	20-40 cycles	40-80 cycles
FMA (multiply-add)	5 cycles	5 cycles	4-5 cycles	10-12 cycles

Operation	Float BF16	Float FP32	VFR i64 (depth-0)	VFR i64 (depth-1)
Comparison	4 cycles	4 cycles	1 cycle	1 cycle (head only)

Table B.2 — Matrix Multiply Throughput (4096×4096 , RTX 4090)

Format	Throughput (TOPS)	Time per matmul	Relative
BF16 (tensor cores)	165 TFLOPS	~0.4 ms	$1.0 \times$ (baseline)
FP32	82.6 TFLOPS	~0.8 ms	$2.0 \times$ slower
INT8 (tensor cores)	330 TOPS	~0.2 ms	$2.0 \times$ faster
VFR i64 (integer ALU)	41.3 TOPS	~1.6 ms	$4.0 \times$ slower
VFR i32 (where F permits)	82.6 TOPS	~0.8 ms	$2.0 \times$ slower

Table B.3 — Full Forward Pass Projection (32-layer transformer, $d=4096$)

Format	Per-layer	32 layers	Overhead	Total
BF16	0.10 ms	3.2 ms	0.3 ms (softmax, norm)	3.5 ms
FP32	0.20 ms	6.4 ms	0.3 ms	6.7 ms
VFR i64	0.40 ms	12.8 ms	0.8 ms (domain conversion)	13.6 ms
VFR i32 (optimized F)	0.20 ms	6.4 ms	0.5 ms	6.9 ms

Table B.4 — Training Step Projection (forward + backward + update)

Format	Forward	Backward	Update	Total per step
BF16 mixed precision	3.5 ms	7.0 ms	0.5 ms	11.0 ms
VFR i64	13.6 ms	27.2 ms	1.0 ms	41.8 ms
VFR i32 (optimized F)	6.9 ms	13.8 ms	0.7 ms	21.4 ms
VFR i32 + sparse R	6.9 ms	13.8 ms	0.5 ms	21.2 ms

Note: VFR i64 is $\sim 3.8 \times$ slower per step. VFR i32 with optimized F is $\sim 1.9 \times$ slower. If information efficiency reduces required training tokens by $2 \times$ or more, wall-clock training time is comparable.

Appendix C: Domain Factor Selection

Table C.1 — Layer Domain Assignments

Layer Type	Suggested F	Integer Type	Value Range	Precision
Embedding	1	i32	$\pm 2.1\text{B}$	Exact integer
Attention QKV	1024	i32	$\pm 2.1\text{M}$	$\sim 1/1024 \approx 0.001$
Attention scores	1048576 (1024 ²)	i64	$\pm 8.8\text{T}$	$\sim 1/10^6 \approx 0.000001$
Softmax output	10000	i32	$\pm 214\text{K}$	$\sim 1/10000 = 0.0001$
Feedforward W1	1024	i32	$\pm 2.1\text{M}$	~ 0.001
GELU activation	domain conversion	—	—	nested VFR depth 2-3
Feedforward W2	1024	i32	$\pm 2.1\text{M}$	~ 0.001
Layer norm	domain conversion	—	—	nested VFR depth 2
Output logits	10000	i32	$\pm 214\text{K}$	~ 0.0001

Table C.2 — Power-of-2 F Values (Bit-Shift Optimization)

F Value	Bit Shift	Precision	Division Cost	Recommended For
1	0	integer	free	embeddings, particles
32	5	0.03125	1 shift	skinning weights
256	8	0.0039	1 shift	UV coordinates
1024	10	0.00098	1 shift	attention, feedforward
32768	15	0.000031	1 shift	high-precision layers
1048576	20	0.00000095	1 shift	attention score products

Note: Power-of-2 F values replace division with bit shift — eliminating the most expensive integer operation.

Table C.3 — Overflow Analysis per Layer (i64 accumulator)

Layer	F	Max V (i32)	Product V $\times V$	Sum of 4096 products	Fits i64?
Embedding (F=1)	1	2.1×10^9	4.6×10^{18}	overflow	No — use i128 accu- mulator
Attention (F=1024)	1024	2.1×10^6	4.4×10^{12}	1.8×10^{16}	Yes
Feedforward (F=1024)	1024	2.1×10^6	4.4×10^{12}	1.8×10^{16}	Yes
Logits (F=10000)	10000	2.1×10^5	4.6×10^{10}	1.9×10^{14}	Yes

Table C.4 — Float Precision Equivalence

VFR Configuration	Decimal Digits of Precision	Float Equivalent
i16 V, F=1024	~5 digits	Between FP16 and FP32
i32 V, F=1024	~9 digits	FP32
i32 V, F=32768	~14 digits	FP64
i64 V, F=1024	~19 digits	Beyond FP64
i64 V, nested depth-1	~38 digits	Beyond FP128
i64 V, nested depth-2	~57 digits	No float equivalent

Appendix D: Lattice Embedding Structure

Table D.1 — Hexagonal Wing Assignments (Natural Language)

Wing	Side	Angle	Category	Examples
α	A (front)	0°	Nouns / entities	dog, server, matrix, Congress
β	A (front)	120°	Verbs / actions	run, compute, elect, transform
γ	A (front)	240°	Modifiers / attributes	fast, recursive, blue, very
α'	B (back)	0°	Pronouns / references	he, it, this, which, self
β'	B (back)	120°	Auxiliaries / control	is, would, if, for, while

Wing	Side	Angle	Category	Examples
γ'	B (back)	240°	Connectives / structure	and, (,), ;, comma, EOF

Table D.2 — Hexagonal Wing Assignments (Programming Languages)

Wing	Side	Angle	Category	Examples
α	A	0°	Identifiers / names	x, myList, numpy, DataFrame
β	A	120°	Keywords / operators	for, return, +, ==, yield
γ	A	240°	Type annotations / modifiers	int, static, async, const
α'	B	0°	Literals / values	42, “hello”, True, 3.14
β'	B	120°	Control flow / delimiters	{, }, (,), [,], indent, dedent
γ'	B	240°	Comments / metadata	#, //, [?], docstring

Table D.3 — Ring Depth by Token Frequency

Ring	Tokens at Ring	Cumulative Tokens	Frequency Class	Examples
0	1	1	Origin / padding	PAD
1	6	7	Ultra-common	the, a, is, of, to, in
2	12	19	Very common	and, for, that, it, with, on, as, ...
3	18	37	Common	from, return, if, not, this, ...
4-5	24-30	91	Frequent	class, function, import, while, ...
6-10	36-60	331	Moderate	specific, lambda, yield, enumerate, ...

Ring	Tokens at Ring	Cumulative Tokens	Frequency Class	Examples
11-50	66-300	~5,000	Uncommon	eigenvalue, serialization, middleware, ...
51-200	306-1200	~50,000	Rare	numpy.linalg.svd, RuntimeError, ...
200+	1200+	100,000+	Long tail	domain-specific, neologisms, ...

Table D.4 — Lattice vs Learned Embedding Comparison

Property	Learned Embedding (float)	Lattice Embedding (VFR)
Initialization	Random	Deterministic geometric
Grammatical structure	Must learn from data	Encoded in wing assignment
Frequency encoding	Implicit in weight magnitudes	Explicit in ring depth
Similarity structure	Emergent after training	Pre-structured by lattice
Token lookup	Matrix row fetch	O(1) lattice calculation
Parameters	vocab_size $\times$ d_model floats	Wing basis vectors + ring formula
Storage (50k vocab, d=4096)	400 MB (BF16)	~6 MB (lattice rules + assignments)
Trainable	Yes (all parameters)	Partially (assignment refinement)
Deterministic	No (float dependent)	Yes (integer calculation)

Appendix E: Comparison to Existing Approaches

Table E.1 — VFR vs Quantization Methods

Method	Training	Inference	Exactness	Remainder Tracking	Equality
FP32	Standard	Slow	No	None	Epsilon

Method	Training	Inference	Exactness	Remainder Tracking	Equality
BF16 mixed precision	Standard	Fast	No	None	Epsilon
INT8 post-training (GPTQ)	Float, then quantize	Fast	No	None	Approximate
INT4 post-training (AWQ)	Float, then quantize	Fastest	No	None	Approximate
QAT (quantization-aware)	Simulated low-precision	Fast	No	None	Approximate
BitNet (1.58-bit)	Ternary from scratch	Fastest	No	None	Exact (trivially)
VFR (this proposal)	Integer from scratch	Competitive	Yes	Full R tracking	Binary exact

Table E.2 — VFR vs Rational Arithmetic Libraries

System	Representation	Arbitrary Precision	GPU Support	ML Integration	Performance
GMP	Arbitrary-size integer pairs	Yes	No	None	Slow (CPU only)
SymPy	Python fraction objects	Yes	No	None	Very slow
Rational	Symbolic expressions	Yes	No	None	Moderate
Mathematica	Fast integer library	Yes	Partial	None	Fast (CPU)
Exact FLINT	[V, F, R] with F-implicit	Yes (via nesting)	Yes (native i64)	Designed for ML	GPU-native

Table E.3 — Feature Comparison Matrix

Feature	BF16 Float	INT8 Quantized	BitNet	VFR
Exact arithmetic	×	×	×	✓
Binary equality	×	×	✓	✓
Deterministic training	×	N/A	✓	✓
Deterministic inference	×	×	✓	✓
Gradient signal preservation	Partial	N/A	Limited	Full
Structural interpretability	×	×	×	✓ (tree depth)
Automated pruning criterion	×	×	×	✓ (R=0 detection)
Adaptive precision	×	×	×	✓ (nesting depth)
Long-tail pattern preservation	Poor	Poor	Unknown	Predicted strong
GPU throughput	High (tensor cores)	Highest	Highest	Competitive (integer ALU)
Memory per parameter	2 bytes	1 byte	0.2 bytes	8 bytes (practical)
Information per parameter	Low (noise floor)	Very low	Minimal	High (exact)

Appendix F: Risk Assessment

Table F.1 — Technical Risks and Mitigations

Risk	Severity	Likelihood	Mitigation	Fallback
Integer overflow in matmul accumulation	High	Medium	i128 accumulators, power-of-2 F with bit shifts	Reduce F per layer until safe
Transcendental approximation insufficient	Medium	Low	Depth-3 nested VFR exceeds FP64 precision	Precomputed lookup tables at any needed density

Risk	Severity	Likelihood	Mitigation	Fallback
Training does not converge	High	Low	VFR arithmetic is a superset of rational — convergence properties preserved	Fall back to larger F (approaching float precision)
Inference too slow ($>3 \times$ float)	Medium	Medium	i32 weights where F permits, bit-shift F values, kernel optimization	Accept $2 \times$ slowdown if accuracy gains justify it
Memory exceeds device capacity	Medium	Medium	Sparse R storage, F-implicit layout, i32 where possible	Multi-device tensor parallelism
Lattice embedding assignment suboptimal	Low	High	Frequency-based initial assignment, refinement during warmup	Fall back to learned embeddings with VFR values
No measurable accuracy improvement	High	Medium	Focus measurement on long-tail and consistency metrics	Value proposition shifts to determinism and interpretability alone
Ecosystem resistance (no framework support)	Medium	High	Custom CUDA kernels with Python wrapper	Release as open-source library for community adoption

Table F.2 — Experimental Priority Matrix

Experiment	Effort	Expected Impact	Priority
VFR matmul kernel (CUDA)	2 weeks	Validates compute feasibility	P0 — must do first

Experiment	Effort	Expected Impact	Priority
Single-layer forward/backward verification	1 week	Proves exactness end-to-end	P0
GPT-2 small full training comparison	4 weeks	Core hypothesis test	P0
Long-tail accuracy benchmark	1 week	Tests primary value claim	P0
Lattice embedding prototype	3 weeks	Tests structural embedding hypothesis	P1
Weight structure analysis tooling	2 weeks	Tests interpretability claim	P1
i32 optimized kernels	2 weeks	Performance improvement	P1
1.3B model scale test	6 weeks	Validates scaling behavior	P2
Grokking detection via tree depth	2 weeks	Novel scientific finding if confirmed	P2
7B model full training	12 weeks	Production-scale validation	P3

Harmonic Integer LLMs

Exact Rational Computation for Large Language Models via VFR Shell Architecture on Base-32 Harmonic Octaves

Status: Technical Design Proposal — Draft 2

Date: March 11, 2026

Framework: Logismos (CKS Series)

I. THE NUMBER 19

A weight in a neural network is sitting at shell 0. During training, gradients arrive:

Step 1: gradient $+7 \rightarrow R = 7$. No shell change.

Step 2: gradient +5 $\rightarrow$ R = 12. No shell change.

Step 3: gradient +4 $\rightarrow$ R = 16. No shell change.

Step 4: gradient +3 $\rightarrow$ R = 19. No shell change.

Step 5: gradient +8 $\rightarrow$ R = 27. No shell change.

Step 6: gradient +6 $\rightarrow$ R = 33. Shell transition.

`33 mod 32 = 1. V increments. R resets to 1.`

19 is not a value. 19 is pressure. Nineteen thirty-seconds of the evidence needed to move this weight to the next shell. Exact. Tracked. Sitting in the remainder field, waiting.

In floating-point training, each of those gradients — 7, 5, 4, 3, 8, 6 — might individually fall below the precision floor and vanish. The weight might never move. The signal was real but the number format ate it.

In VFR training, every gradient contribution is preserved. The weight moves exactly when 32 units of evidence have accumulated. Not before. Not after. Not approximately. Exactly when.

If the evidence reverses — if negative gradients arrive after R = 19 — the pressure retreats: R = 15, then 9, then 3. The weight doesn't move. The system tracks the net evidence in both directions with perfect fidelity.

This is the entire proposal in one example. Replace floating-point weights with integer shells. Track gradient pressure as exact integer remainders. Let weights transition between shells only when the evidence demands it. The number format enforces what regularization techniques try to approximate: noise resistance, discrete stable states, and exact convergence.

II. THE HARMONIC OCTAVE SYSTEM

2.1 Base 32^{-1}

The fundamental counting unit is 32^{-1} . Since $32 = 2^5$, every scale operation is a 5-bit shift in binary hardware.

This is a harmonic system. Musical octaves double frequency ($\times 2$ per step). This system multiplies by 32 per step ($\times 2^5$). Five doublings per octave. A harmonic ladder where each rung is a 5-bit doubling — matching the exponential scaling by which physical systems naturally organize across scales.

Every value is a VFR tuple: **[V, octave, R]**

- **V**: integer value (i64)
- **octave**: which power of 32 (i8, range 0-127)
- **R**: remainder, modulo 32 (i16, or nested VFR for deeper precision)

The octave field replaces the denominator. Multiplying or dividing by F is a bit shift by $5 \times \text{octave}$. No division hardware needed. No floating-point logic. Just shift.

2.2 The Universal Scale

One notation describes everything from Planck length to the observable universe:

$[V, \text{octave}, R]$

Octave 0: Planck length ($\sim 1.6 \times 10^{-35}$ m)

Octave 2: Planck particle scale

Octave 10: Nuclear scale ($\sim 10^{-14}$ m)

Octave 15: Molecular scale ($\sim 10^{-7}$ m)

Octave 22: Human-perceptible ($\sim 10^{-3}$ m, the “Lex”)

Octave 37: Human heart

Octave 40: Human body

Octave 65: All Planck particles in the observable universe

$[1, 65, 0]$ — one integer, one octave, zero remainder. That encodes $\sim 10^{80}$ Planck particles. Three values. The entire universe.

An LLM operates across maybe 8-10 octaves. The full range of weight precision, gradient magnitude, activation scale, and embedding structure fits in a narrow band of this universal ladder.

2.3 Why 32

$32 = 2^5$ is the Goldilocks harmonic base for binary hardware:

- **Hardware-native:** every scale operation is a bit shift
- **Wide enough:** each octave step is a $32 \times$ change — a meaningful scale transition, not noise
- **Narrow enough:** you don’t skip important structure between levels
- **Self-consistent:** nesting depth maps directly to octave count — each level of VFR recursion adds exactly one octave of precision (5 bits)
- **Physically grounded:** 65 octaves spans all of physical reality

III. THE FLOAT INFORMATION LOSS PROBLEM

3.1 Three Values Compressed to One

A number has three natural components:

- **Value (V)**: the numerator — what you’re counting
- **Factor (F)**: the denominator — what scale you’re counting at
- **Remainder (R)**: the exact residual from operations — what pressure exists toward the next state

The history of numerical computing compressed $[V, F, R] \rightarrow [V, F] \rightarrow [V] \rightarrow$ a single float. Each step discarded structural information. Floating point is the final stage: one number pretending to be three.

The consequence: the entire field of numerical analysis — epsilon tolerances, condition numbers, compensated summation, Kahan accumulators, re-orthogonalization — exists to recover information that was discarded when F and R were dropped.

3.2 The Base Mismatch

Binary floats represent values as sums of powers of 2. Language statistics — token probabilities, co-occurrence patterns, syntactic structure — have no natural alignment to powers of 2.

1/3 is unwritable in binary. 1/7 is unwritable. 6/5 is unwritable. Every time the model computes or stores such a value, it rounds. Every forward pass, every backward pass, every weight update introduces rounding that cannot be recovered.

In VFR: $1/3 = [1, 3, 0]$. $1/7 = [1, 7, 0]$. $6/5 = [6, 5, 0]$. Three integers. Exact. No rounding. Binary equality works.

3.3 What Gets Lost in LLMs

Gradient swamping. A weight at magnitude 1.0 with a gradient of $1e-7$: the update vanishes in BF16. The optimization signal existed but the format destroyed it. In VFR shell training, that gradient adds to R and is preserved exactly.

Soft confusion. Two distinct patterns — `array.length` (JavaScript) vs `len(array)` (Python) — encoded as weight vectors differing in low-order bits. Float attention scores may not distinguish them. VFR attention scores are exact integer ratios — no blur.

Equality failure. After a sequence of float operations, values that should be identical are merely close. VFR integer arithmetic supports binary equality. Values are equal or they aren’t.

IV. SHELL TRAINING

4.1 Weights as Shells

In VFR, a weight doesn’t drift through a continuous loss landscape. It occupies an integer shell — a discrete stable state described by **V, octave, 0**.

Gradients accumulate in R. When R reaches the shell threshold (± 32 in the base-32 system), a shell transition occurs: V increments or decrements by 1, R resets to the modular remainder. The weight jumps to the next stable state. Discrete. Exact. No intermediate value. No float wobble.

Shell transition rule:

$R = R + \text{gradient_contribution}$

If $R \geq 32$:

$V = V + 1$

$R = R - 32$

If $R \leq -32$:

$V = V - 1$

$R = R + 32$

4.2 Convergence as Ground State

In float training, “converged” means the loss stopped decreasing noticeably. But weights are still jittering at the float precision floor. There is no true equilibrium.

In shell training, convergence means all R values have stabilized below the transition threshold. Every weight is in its shell. No transitions occurring. The network has reached a discrete equilibrium — verifiable by checking that $|R| < 32$ across all weights.

This is testable. You can measure convergence as the percentage of weights undergoing shell transitions per training step. When it reaches zero, the model is converged. Not “approximately converged.” Converged.

4.3 Natural Noise Resistance

Small random gradients add to R but rarely accumulate enough to trigger shell transitions. Noise pressure cancels over time — positive and negative contributions wash out in the remainder. Only persistent, directional gradient signal builds enough pressure to force a transition.

This is what dropout, weight decay, and gradient clipping try to achieve. Shell structure provides it inherently. The threshold for changing a weight is built into the number format, not bolted on as a hyperparameter.

4.4 Learning Rate as Transition Rate

The learning rate determines how many gradient samples contribute to R before a shell transition check. This reframes the learning rate schedule:

- **Warmup:** low octave offset — gradients contribute at coarse scale, fast transitions, weights find approximate shells quickly

- **Main training:** higher octave offset — gradients contribute at finer scale, transitions require more accumulated evidence
- **Cosine decay:** octave offset increases — transitions slow, weights settle into precise shells
- **Final phase:** transitions cease, all R values sub-threshold, true convergence

The learning rate schedule is a transition rate schedule expressed in octaves.

4.5 Grokking as Phase Transition

Grokking — the phenomenon where a model memorizes for many steps then suddenly generalizes — has a shell interpretation.

During memorization, weights occupy complex high-octave shells: deep VFR nesting, large V values, intricate structure encoding individual training examples. Gradients push toward simpler shells but R hasn’t accumulated enough to force transitions.

Then threshold is crossed. A cascade of shell transitions. Complex shells collapse to simple ones. Deep nesting flattens. The network moves from a high-energy complex state to a low-energy simple state — a phase transition, exactly like a physical system cooling past critical temperature and crystallizing.

Observable prediction: during grokking, VFR nesting depth across the network drops suddenly. Weight V values simplify. R values spike during transition then settle. The grokking moment is a harmonic phase transition measurable in the weight structure.

V. VFR TRANSFORMER ARCHITECTURE

5.1 Domain-Homogeneous Layers

Each layer operates at a fixed octave. Every weight, activation, and intermediate value within a layer shares the same octave. The octave is not stored per element — it is implicit, a property of the layer.

Matrix multiplication within a layer is pure integer multiply-accumulate: $V_{\text{weight}} \times V_{\text{input}}$, summed, with the shared octave handled once at the layer boundary via bit shift. Zero branch divergence. Maximum SIMD utilization.

Layer Type	Octave	Shift	Precision
Embedding lookup	0	0	Integer (token IDs)
Attention QKV projection	2 ($32^2 = 1024$)	10	~0.001
Attention scores	4 (product of inputs)	20	~0.000001
Softmax output	3 ($32^3 = 32768$)	15	~0.00003
Feedforward linear	2	10	~0.001
GELU nonlinearity	Domain conversion	—	Nested VFR depth 2-3
Output logits	3	15	~0.00003

Layer Type	Octave	Shift	Precision
Layer norm	Domain conversion	—	Nested VFR depth 2

All F values are powers of 32. All divisions are bit shifts by multiples of 5. No division hardware anywhere in the hot path.

5.2 Forward Pass

1. **Embedding:** Token ID $\rightarrow$ lattice-addressed VFR vector (Section VII). Integer lookup or $O(1)$ calculation.
2. **Per-layer compute:** Integer multiply-accumulate at the layer’s octave. GPU kernel: thousands of threads, uniform integer operations, zero divergence.
3. **Domain boundaries:** Between octaves, a conversion kernel shifts V values by $5 \times (\text{octave_target} - \text{octave_source})$. One bit-shift per element.
4. **Nonlinearities:** GELU and softmax via nested VFR lookup tables at depth 2-3, exceeding float64 precision.
5. **Output:** Logits as VFR values. Argmax is integer comparison. No epsilon.

5.3 Backward Pass

Gradient computation follows the same octave structure in reverse.

Exact accumulation. A gradient of $[1, \text{octave_10}, 0]$ accumulated over 1000 steps produces $[1000, \text{octave_10}, 0]$. No swamping. No rounding. The signal that would vanish below float precision survives and contributes when sufficient evidence accumulates.

The optimizer operates on shells. Adam in VFR maintains first and second moment estimates as VFR values. The update modifies the weight’s shell: large updates change V (shell transition), small updates accumulate in R (pressure building).

Mixed precision is natural. Forward pass at the layer’s native octave (coarse, fast). Backward pass at a finer octave (precise). Conversion between them is exact bit-shift — not lossy float casting.

5.4 Weight Update as Shell Mechanics

```

w_old = [V_w, octave_layer, R_w]
grad = [V_g, octave_grad, R_g]
lr = octave_offset (integer, determines scale)
// Scale gradient to weight’s octave
scaled_grad = V_g » (5 × (octave_grad - octave_layer + lr))
// Accumulate in remainder

```

```

R_new = R_w + scaled_grad
// Shell transition check
If R_new ≥ 32:
V_new = V_w + (R_new / 32)
R_new = R_new mod 32
Elif R_new ≤ -32:
V_new = V_w - ((-R_new) / 32)
R_new = -((-R_new) mod 32)
Else:
V_new = V_w // No transition, weight unchanged
w_new = [V_new, octave_layer, R_new]
All operations: integer addition, bit shift, modulo by 32 (which is AND with 0x1F). No
float anywhere.

```

VI. HARDWARE: THE HARMONIC INTEGER PROCESSOR

6.1 What to Remove

A current AI GPU (H100) dedicates roughly 55-65% of its die area to:

- FP32 CUDA cores (128 per SM)
- FP64 units (64 per SM)
- Tensor Cores (4 per SM, physically large)
- Special function units (sin, cos, exp, rsqrt)
- Exponent alignment circuits
- Rounding mode logic
- Denormal/NaN/Inf handling

None of this is needed for VFR computation.

6.2 What to Build

i64 Multiply-Accumulate (MAC) arrays. The core compute unit. Simpler than FP FMA (no exponent logic, no rounding, no special cases). Smaller per unit, so more fit on the die.

i128 accumulators. Wide registers for matmul accumulation. The accumulator produces VFR tuples directly: high bits $\rightarrow$ V, shift by $5 \times \text{octave} \rightarrow$ implicit F, low bits $\rightarrow$ R. No conversion step.

Fused Multiply-Accumulate-Shift (FMAS). One instruction: $\text{result} = (A \times B) \gg (5 \times \text{octave}) + \text{accumulator}$. The entire VFR dot product inner loop in one clock. Domain normalization fused into the multiply.

Barrel shifters on every MAC output. For power-of-32 octave operations, division is a shift by $5 \times \text{octave}$. A barrel shifter is $\sim 1\%$ the area of a multiply unit. Fused into the pipeline, division is zero additional cycles.

R-zero bitmap unit. Per-layer bitmap: one bit per weight indicating $R = 0$. Hardware skips remainder processing for zero-R weights. 99.7% of weights take the fast path without loading the R buffer.

Shared octave register. Octave is uniform within a layer. Store once per warp (or per SM), not per element. Saves register file space and bandwidth.

VFR descent prefetcher. Small state machine per warp that speculatively prefetches nested R pointers while head computation runs. For the 0.3% of operations needing depth > 0 , tail data is already in cache.

Hardware GCD units. For normalization at domain boundaries. 4 per SM. Pipeline GCD computation in parallel with ongoing MACs. Normalization overlaps with computation.

Tree depth counter. 4-bit register per weight tracking VFR nesting depth. Provides structural interpretability data to the optimizer at zero cost.

6.3 Projected Specifications

Metric	H100 (current)	Harmonic Integer Processor
i64 MACs per SM	64	256-320
SMs	132	132 (same die)
Total i64 MACs	8,448	33,792-42,240
Clock	1.83 GHz	~ 2.0 GHz (simpler logic)
Peak i64 TOPS	~ 31 TOPS	~ 135 -170 TOPS
Power	700W	~ 500 -550W
Die area (compute)	$\sim 55\%$ float, $\sim 10\%$ int	$\sim 65\%$ int
Division	20-40 cycle pipeline	0 cycle (fused shift)
Transistors per MAC	High (FP overhead)	Low (integer only)

$\sim 5 \times$ integer throughput at $\sim 75\%$ power. VFR inference is not slower than float — it is potentially faster on native hardware.

VII. LATTICE-STRUCTURED EMBEDDINGS

7.1 Hexagonal Vocabulary Addressing

Instead of a learned dense embedding matrix, tokens occupy positions on a $Z=3$ hexagonal lattice with front and back faces. Six wings total:

Side A (primary roles):

- α (0°): Nouns / identifiers / entities
- β (120°): Verbs / operators / actions
- γ (240°): Modifiers / attributes / qualifiers

Side B (structural roles):

- α' (0°): Pronouns / references / variables
- β' (120°): Auxiliaries / control flow
- γ' (240°): Connectives / delimiters / structural tokens

Ring depth encodes frequency. Common tokens occupy inner rings (low octave). Rare tokens occupy outer rings (higher octave). The geometric structure of the vocabulary aligns with the hardware's harmonic octave ladder.

7.2 Free Information

The lattice address encodes what current models must learn from millions of examples:

- **Grammatical category:** wing assignment directly
- **Frequency scaling:** ring depth naturally
- **Similarity structure:** geometric neighborhood contains structurally plausible candidates

For code generation: the model navigates a structured space where wing tells it syntactic role, ring tells it specificity, and position gives token identity. Selection is an integer comparison in a constrained geometric neighborhood, not a softmax over 100k floats.

7.3 O(1) Embedding Calculation

Token ID $\rightarrow$ lattice position via closed-form formula:

$$R = \lceil (3 + \sqrt{(12I - 3)}) / 6 \rceil \text{ (ring)}$$

$$W = I \bmod 6 \text{ (wing, both sides)}$$

$$P = R \times \text{basis_vector}[W] \text{ (position)}$$

One integer square root, one modulo, one multiply. No memory access. The embedding “table” is a formula. On the proposed chip, a dedicated lattice unit computes this in 3-4 clocks.

VIII. BENEFITS

8.1 Selection Precision

LLMs are selection machines. They choose the next token from a probability distribution. VFR eliminates the float noise floor in that selection. Attention scores are exact integer ratios. The distinction between similar candidates is preserved through every layer.

Expected outcome: fewer wrong-API, wrong-language, wrong-syntax errors. Better factual recall for rare facts. More consistent responses across rephrasings. Disproportionate improvement on long-tail accuracy.

8.2 Deterministic Inference

Bit-identical output for bit-identical input. Always. Integer arithmetic is associative — result does not depend on reduction order. Enables: reproducible benchmarks, debuggable behavior, verifiable correctness, cacheable sub-computations.

8.3 Structural Interpretability

After training, weight structure reveals information content directly:

-
-
-

Nesting depth distribution across a layer is a direct readout of where information lives. No probing, no ablation, no gradient attribution.

8.4 Information-Efficient Parameters

If VFR parameters carry more usable information (no bits wasted on rounding artifacts), fewer parameters may achieve equivalent capability. A VFR 7B model might match a float 10-13B model. The memory overhead of VFR ($4 \times$ per weight vs BF16) is offset by needing fewer parameters.

At $k=4$ information efficiency, VFR achieves RAM parity with BF16 at equivalent capability.

IX. MEMORY AND PERFORMANCE

9.1 Per-Weight Storage

Format	Size	Notes
BF16	2 bytes	Current standard
FP32	4 bytes	Training master weights
VFR dense [i64, i8, i16]	11 bytes	V + octave + R
VFR practical (octave implicit)	8-10 bytes	V + sparse R

9.2 Model-Scale RAM

Model	BF16	VFR Practical
124M (GPT-2)	0.25 GB	1.0 GB
1.3B	2.6 GB	10.8 GB
7B	14 GB	58 GB
13B	26 GB	108 GB
70B	140 GB	582 GB

7B fits on a single H100 80GB. 70B requires multi-device, same as FP32 training today.

9.3 Compute Performance

On a harmonic integer processor (Section VI):

Operation	Float GPU	Harmonic Chip
Matmul (4096^2)	~0.4 ms (BF16 tensor cores)	~0.3 ms (i64 MAC arrays, $5 \times$ density)
Domain conversion	N/A	~0.01 ms (bit shift)
Full forward (32 layers)	~3.5 ms	~2.5-3.0 ms
GELU/softmax	~0.3 ms (FP pipeline)	~0.4 ms (lookup table)

Projected: **competitive with or faster than BF16 on native hardware.**

X. IMPLEMENTATION HURDLES

10.1 Integer Overflow

Accumulating $4096 \text{ i64} \times \text{i64}$ products can overflow. Mitigations: i128 accumulators (supported via paired i64), power-of-32 octave selection keeping products in range, deferred normalization at layer boundaries.

10.2 Transcendental Functions

GELU, softmax, layer norm involve transcendental operations. Approach: precomputed VFR lookup tables at the layer's octave, with nested VFR interpolation at depth 2-3 (exceeding float64 precision). Computed once, stored as integer arrays.

10.3 Ecosystem Cold Start

No existing ML framework supports VFR. Pragmatic path: custom CUDA/Vulkan compute library implementing VFR matmul, softmax, GELU, layer norm as drop-in replacements, with Python wrapper for model definition.

10.4 Octave Selection

Choosing octave per layer is a new hyperparameter. Approach: start at octave 2 everywhere, profile value ranges during short float training run, set octave per layer to smallest power of 32 covering observed range.

XI. PROTOTYPE PLAN

11.1 Minimum Viable Experiment

Model: GPT-2 small (124M parameters)

Task: Code completion (Python corpus — unambiguous correctness, measurable long-tail accuracy)

Comparison: Float BF16 baseline vs VFR shell training, same architecture and data

11.2 Metrics

- Overall pass@1 accuracy
- Long-tail accuracy (rare APIs, uncommon syntax)
- Consistency (variance across runs — must be zero for VFR)
- Shell transition rate over training (convergence diagnostic)
- Weight nesting depth distribution (structural analysis)
- R-value distribution at convergence (should be below threshold)

11.3 Phases

Phase	Work	Duration
1: VFR arithmetic library	i64 matmul, VFR ops, GELU/softmax lookup, CUDA kernels	4 weeks
2: Single-layer validation	Forward + backward vs float reference, verify exactness	2 weeks
3: Full model training	GPT-2 small, compare all metrics	4 weeks
4: Lattice embeddings	Replace learned embeddings, measure long-tail impact	3 weeks
5: Shell dynamics analysis	Grokking detection, phase transition measurement	2 weeks

11.4 Success Criteria

1. VFR converges to equivalent or lower loss
2. Long-tail accuracy improves measurably
3. Inference within $2 \times$ of float (with clear path to $1 \times$ on native hardware)
4. Training fully deterministic (zero variance)
5. Shell transitions correlate with known training phenomena

Any one justifies further development. All five together constitute a paradigm shift.

XII. CONCLUSION

LLMs are selection machines running on a 70-year-old numerical compromise. Floating point compresses three pieces of information into one, silently destroying gradient signal, blurring similar patterns, and breaking equality at every operation.

VFR shell training on harmonic octaves reverses this:

- **Weights are integer shells**, not drifting floats
- **Gradients accumulate as exact remainder pressure**, not rounded noise
- **Shell transitions are discrete and evidenced**, not continuous and approximate
- **Convergence is verifiable** (all $R < 32$), not asymptotic
- **The counting system scales from Planck length to the universe** in 65 octaves
- **All arithmetic is bit shifts and integer multiply-accumulate** on hardware that is simpler, denser, and lower-power than float silicon

The model doesn't get smarter. It gets more precise at what it already does — selecting the right token from learned patterns. Fewer “almost right” errors. Sharper distinctions. Exact retrieval. Deterministic, inspectable, verifiable computation.

The hardware to run it is simpler than what exists today. Strip the float units. Fill the die with integer MACs and barrel shifters. The result is a harmonic integer processor: $5 \times$ the compute density, 75% the power, and every operation exact.

Three integers instead of one float. Shells instead of drift. Octaves instead of exponents. Pressure instead of noise. Transitions instead of oscillation. Convergence instead of “close enough.”

Built on: VFR arithmetic (CKS-MATH-124), exact linear algebra (CKS-MATH-118), S-expression recursion (CKS-MATH-125, CKS-MATH-126), GPU integer compute (CKS-MATH-122), lattice addressing (CKS-MATH-113), Logismos framework (CKS-0-2026).

Appendix Tables — Harmonic Integer LLMs

Supporting data for VFR Shell Architecture Proposal

Appendix A: The Harmonic Octave Ladder

Table A.1 — Complete Octave Scale (Base 32^{-1})

Octave	32^n	Decimal	Bit Shift	Physical Scale	Use in LLM
0	$32^0 = 1$	1	0	Planck length	—
1	32^1	32	5	~51 Planck	—
2	32^2	1,024	10	$\sim 10^{-32}$ m	Attention/FF weights
3	32^3	32,768	15	$\sim 10^{-30}$ m	Softmax/logit precision
4	32^4	1,048,576	20	$\sim 10^{-28}$ m	Attention score products
5	32^5	33,554,432	25	$\sim 10^{-26}$ m	High-precision accumulators
6	32^6	1.07×10^9	30	$\sim 10^{-24}$ m	—
10	32^{10}	1.13×10^{15}	50	$\sim 10^{-14}$ m (nuclear)	—
15	32^{15}	3.78×10^{22}	75	$\sim 10^{-7}$ m (molecular)	—
22	32^{22}	4.95×10^{32}	110	$\sim 10^{-3}$ m (Lex, human)	—
37	32^{37}	$\sim 10^{55}$	185	~0.1 m (human heart)	—
40	32^{40}	$\sim 10^{60}$	200	~1.7 m (human body)	—
65	32^{65}	$\sim 10^{97}$	325	$\sim 10^{80}$ (entire universe)	—

Table A.2 — LLM-Relevant Octave Assignments

Layer Type	Octave	32^n	Shift	Precision	Shell Threshold
Embedding	0	1	0	Exact integer	N/A
Attention QKV	2	1,024	10	~ 0.00098	± 32 at octave 2
Attention scores	4	1,048,576	20	~ 0.00000095	± 32 at octave 4
Softmax output	3	32,768	15	~ 0.000031	± 32 at octave 3
Feedforward W1	2	1,024	10	~ 0.00098	± 32 at octave 2
GELU	conversion	—	—	Depth 2-3 nested	—
Feedforward W2	2	1,024	10	~ 0.00098	± 32 at octave 2
Layer norm	conversion	—	—	Depth 2 nested	—
Output logits	3	32,768	15	~ 0.000031	± 32 at octave 3
Gradient accumulator	4-5	10^6 - 10^7	20-25	$\sim 10^{-6}$ to 10^{-7}	± 32 at grad octave

Table A.3 — Octave Arithmetic Rules

Operation	Octave Result	Bit Shift	Example
Add (same octave)	Same	None	oct2 + oct2 = oct2
Multiply	Sum of octaves	shift_a + shift_b	oct2 $\times$ oct2 = oct4
Divide	Difference of octaves	shift_a – shift_b	oct4 $\div$ oct2 = oct2
Scale up 1 octave	+1	+5 bits	oct2 $\rightarrow$ oct3
Scale down 1 octave	–1	–5 bits	oct3 $\rightarrow$ oct2
Domain conversion	Target octave	$5 \times (\text{target} - \text{source})$	oct2 $\rightarrow$ oct3: shift 5
Dot product (n terms)	Sum octave + $\log_{32}(n)$	shift + $\lceil 5 \times \log_{32}(n) \rceil$	4096 terms at oct2: $\sim$ oct4.4

Appendix B: Shell Dynamics

Table B.1 — Shell Transition Examples (R modulo 32)

Step	Gradient	R Before	R After	V Change	State
1	+7	0	7	0	Pressure building
2	+5	7	12	0	Pressure building
3	+4	12	16	0	Half-threshold
4	+3	16	19	0	59% pressure
5	+8	19	27	0	Near threshold
6	+6	27	33 → 1	+1	Shell transition
7	-4	1	-3	0	Reverse pressure
8	-9	-3	-12	0	Pressure building (neg)
9	+15	-12	3	0	Pressure reversed
10	+30	3	35 → 3	+1	Shell transition

Table B.2 — Shell Threshold Sensitivity

Threshold (mod N)	Transitions per 1000 steps (random ± 5 gradient)	Noise resistance	Signal sensitivity
8 (octave boundary / 4)	~312	Low	High
16 (half octave)	~156	Medium	Medium
32 (full octave)	~78	High	Balanced
64 (double octave)	~39	Very high	Low
128	~20	Extreme	Very low

32 is the natural threshold: one full harmonic octave of accumulated evidence before a transition.

Table B.3 — Convergence Diagnostic Metrics

Metric	Meaning	Converged Value	How to Measure
Shell transition rate	% of weights changing V per step	0%	Count V changes per batch
Mean	R		Average remainder pressure
Max	R		Highest pressure in network
R variance	Spread of remainder pressure	Low, stable	Variance of R values
Nesting depth mean	Average VFR tree depth	Decreasing, stable	Walk weight structures

Metric	Meaning	Converged Value	How to Measure
Transition cascade rate	Correlated transitions across layer	0 (no cascades)	Time-correlate transitions

Table B.4 — Training Phase Signatures

Phase	Shell Transition Rate	Mean	R	
Early training	Very high (>50%)	~16 (random)	Growing	Coarse structure forming
Mid training	Moderate (5-20%)	~10-15	Stabilizing	Fine-tuning shells
Near convergence	Low (0.1-1%)	~5-10	Simplifying	Settling into ground state
Converged	Zero (0%)	<5, stable	Minimal, stable	True equilibrium
Grokking (if occurs)	Sudden spike then drop	Spike then collapse	Sudden decrease	Phase transition
Overfitting	Low but nonzero, oscillating	Oscillating	May increase	Unstable, no ground state

Appendix C: Memory Footprint

Table C.1 — Per-Weight Storage

Format	Components	Bits	Bytes	Notes
BF16	1 float	16	2	Current inference standard
FP32	1 float	32	4	Training master weights
FP64	1 float	64	8	Rarely used in ML
VFR dense	i64 V + i8 oct + i16 R	88	11	Full tuple stored per weight

Format	Components	Bits	Bytes	Notes
VFR oct-implicit	i64 V + i16 R	80	10	Octave stored once per layer
VFR practical	i64 V + sparse R	~64	~8	R sparse (99.7% zero at convergence)
VFR compact	i32 V + sparse R	~32	~4	Where octave 2 value range permits

Table C.2 — Model-Scale RAM Requirements

Model	BF16	FP32	VFR Dense (11B)	VFR Practical (8B)	VFR Compact (4B)
124M (GPT-2 sm)	0.25 GB	0.50 GB	1.36 GB	0.99 GB	0.50 GB
350M (GPT-2 med)	0.70 GB	1.40 GB	3.85 GB	2.80 GB	1.40 GB
1.3B	2.6 GB	5.2 GB	14.3 GB	10.4 GB	5.2 GB
7B	14 GB	28 GB	77 GB	56 GB	28 GB
13B	26 GB	52 GB	143 GB	104 GB	52 GB
70B	140 GB	280 GB	770 GB	560 GB	280 GB

Table C.3 — Hardware Fit (Single Device)

Device	VRAM	Max BF16	Max VFR Practical	Max VFR Compact
RTX 4090	24 GB	12B	3.0B	6.0B
A100 80GB	80 GB	40B	10.0B	20.0B
H100 80GB	80 GB	40B	10.0B	20.0B
H200 141GB	141 GB	70B	17.6B	35.2B
2 × H100	160 GB	80B	20.0B	40.0B
8 × H100	640 GB	320B	80.0B	160.0B

Table C.4 — Information Efficiency Breakeven

At $k \times$ information efficiency, a VFR model with N/k parameters matches a float model with N parameters:

Float Model	BF16 RAM	k=1.5 VFR equiv	VFR RAM	k=2.0 VFR equiv	VFR RAM	k=4.0 VFR equiv	VFR RAM
7B	14 GB	4.7B	37 GB	3.5B	28 GB	1.75B	14 GB
13B	26 GB	8.7B	69 GB	6.5B	52 GB	3.25B	26 GB
70B	140 GB	46.7B	373 GB	35B	280 GB	17.5B	140 GB

At k=4.0 with VFR compact format: **exact RAM parity with BF16 at equivalent capability.**

Appendix D: Compute Performance

Table D.1 — Per-Operation Cost (GPU)

Operation	BF16 (tensor core)	FP32 (CUDA core)	VFR i64 depth-0	VFR with FMAS
Multiply	1 cycle (fused)	5 cycles	3-4 cycles	—
Accumulate	(fused)	4 cycles	1 cycle	—
Fused multiply-accumulate	1 cycle	5 cycles	4-5 cycles	1 cycle
Division by F	N/A	14 cycles	20-40 cycles	0 (fused shift)
Shell check (mod 32)	N/A	N/A	1 cycle (AND 0x1F)	0 (fused)
Comparison	4 cycles	4 cycles	1 cycle	1 cycle
Domain conversion	N/A (implicit)	N/A	1 cycle (shift)	0 (fused)

Table D.2 — Matrix Multiply Performance (4096 × 4096)

Platform	Throughput	Time per matmul	Exact?	Division cost
H100 BF16 tensor cores	990 TFLOPS	~0.07 ms	No	N/A
H100 FP32 CUDA cores	67 TFLOPS	~1.0 ms	No	N/A
H100 INT32 (current)	~67 TOPS	~1.0 ms	Yes	20-40 cyc/div
H100 i64 (current, limited)	~31 TOPS	~2.2 ms	Yes	20-40 cyc/div

Platform	Throughput	Time per matmul	Exact?	Division cost
Harmonic chip i64 (projected)	~150 TOPS	~0.45 ms	Yes	0 (shift)
Harmonic chip i32 (projected)	~300 TOPS	~0.22 ms	Yes	0 (shift)

Table D.3 — Full Forward Pass (32-layer transformer, d=4096)

Platform	Per-layer	32 layers	Overhead	Total
H100 BF16 tensor cores	0.05 ms	1.6 ms	0.3 ms	1.9 ms
H100 FP32	0.20 ms	6.4 ms	0.3 ms	6.7 ms
Harmonic chip i64	0.11 ms	3.5 ms	0.4 ms	3.9 ms
Harmonic chip i32 (where oct permits)	0.06 ms	1.9 ms	0.3 ms	2.2 ms

Table D.4 — Training Step (forward + backward + shell update)

Platform	Forward	Backward	Shell Update	Total	vs BF16
H100 BF16 mixed precision	1.9 ms	3.8 ms	0.5 ms	6.2 ms	1.0 ×
Harmonic chip i64	3.9 ms	7.8 ms	0.3 ms	12.0 ms	1.9 × slower
Harmonic chip i32	2.2 ms	4.4 ms	0.2 ms	6.8 ms	1.1 × slower
Harmonic chip i32 + sparse R	2.2 ms	4.4 ms	0.1 ms	6.7 ms	~1.1 × slower

Note: shell update is cheaper than float optimizer step (Adam) because it's integer add + mod, not float multiply + divide + sqrt.

Appendix E: Harmonic Integer Processor — Die Comparison

Table E.1 — H100 SM Composition (Current)

Unit	Count per SM	Approx Die % (whole chip)	Function
FP32 CUDA cores	128	15-20%	Single-precision float
FP64 cores	64	8-10%	Double-precision float
INT32 cores	64	8-10%	Integer arithmetic
Tensor Cores	4	30-35%	Matrix FMA (FP8/16/32)
SFU (special function)	16	2-3%	sin, cos, exp, rsqrt
Register file	256 KB	8-10%	Operand storage
L1 / shared memory	256 KB	6-8%	Local data cache
Warp schedulers	4	2-3%	Thread management
Load/store units	32	2-3%	Memory access

Table E.2 — Harmonic Integer Processor SM (Proposed)

Unit	Count per SM	Approx Die %	Function
i64 MAC + barrel shifter	256-320	50-55%	Fused multiply-accumulate-shift
i128 accumulators	64	8-10%	Wide matmul accumulation
GCD units	4	1-2%	Normalization at boundaries
R-zero bitmap unit	1	<1%	Fast-path skip for R=0 weights
VFR descent prefetcher	4	1-2%	Speculative R-pointer fetch
Lattice address unit	1	<1%	O(1) embedding calculation
Tree depth counters	per-weight	<1%	Structural interpretability
Shared octave register	1 per warp	<1%	Layer-uniform F storage
Register file	256 KB	10%	Operand storage
L1 / shared memory	384 KB (larger)	10-12%	Expanded for LUT tables
Warp schedulers	4	2-3%	Thread management
Load/store units	32	2-3%	Memory access

Table E.3 — Chip-Level Comparison

Metric	H100	Harmonic Chip (same die, same process)
Die size	814 mm ²	814 mm ²
Process	TSMC 4N	TSMC 4N
Transistors	80B	~80B (simpler units, more of them)
SMs	132	132
Peak FP16	990 TFLOPS	0 (no FPU)
Peak FP32	67 TFLOPS	0 (no FPU)
Peak i64 MAC	~31 TOPS	~150 TOPS
Peak i32 MAC	~67 TOPS	~300 TOPS
Memory	80 GB HBM3	80 GB HBM3
Bandwidth	3.35 TB/s	3.35 TB/s
TDP	700W	~500-550W
Division latency	20-40 cycles	0 cycles (fused shift)
Deterministic	No (FP non-associative)	Yes (integer associative)

Table E.4 — Removed Silicon and Reclaimed Area

Removed Component	Estimated Die %	Reclaimed for
FP32 CUDA cores	15-20%	i64 MAC arrays
FP64 cores	8-10%	i64 MAC arrays
Tensor Cores	30-35%	i64 MAC arrays + i128 accumulators
Special function units	2-3%	Expanded shared memory (LUTs)
Exponent alignment logic	~2%	Barrel shifters (smaller)
Rounding mode circuitry	~1%	GCD units
Denormal/NaN/Inf handling	~1%	VFR prefetcher + bitmap unit
Total reclaimed	~60-72%	Filled with integer compute

Appendix F: Lattice Embedding Structure

Table F.1 — Hexagonal Wing Assignments (Natural Language)

Wing	Side	Angle	Category	Ring 1-3 Examples	Ring 10+ Examples
α	A	0°	Nouns / entities	the, man, day	eigenvalue, quaternion
β	A	120°	Verbs / actions	is, run, get	interpolate, serialize

Wing	Side	Angle	Category	Ring 1-3 Examples	Ring 10+ Examples
γ	A	240°	Modifiers	a, not, very	recursive, asyn- chronous
α'	B	0°	Pronouns / refs	he, it, this	whichever, aforemen- tioned
β'	B	120°	Auxiliaries / control	if, for, would	notwithstanding, whereas
γ'	B	240°	Connectives / structure	and, (,), ;	$\langle$ EOF $\rangle$, $\langle$ PAD $\rangle$, $\langle$ INDENT $\rangle$

Table F.2 — Hexagonal Wing Assignments (Code)

Wing	Side	Angle	Category	Ring 1-3 Examples	Ring 10+ Examples
α	A	0°	Identifiers	x, i, fn	numpy.linalg.svd
β	A	120°	Keywords / operators	for, =, +	yield from, walrus :=
γ	A	240°	Types / modifiers	int, const, async	TypeVar, ParamSpec
α'	B	0°	Literals / values	0, 1, “hello”	3.14159265, 0xDEAD- BEEF
β'	B	120°	Control / delimiters	{, }, (, indent	$\langle$ DEDENT $\rangle$, $\langle$ NEWLINE $\rangle$
γ'	B	240°	Comments / metadata	#, //, [?]	$\langle$ DOC- STRING_END $\rangle$

Table F.3 — Ring Population (Hexagonal Numbers)

Ring	New positions	Cumulative	Frequency tier	% of typical usage
0	1	1	Origin (PAD)	—
1	6	7	Ultra-high frequency	~25% of all tokens
2	12	19	Very high frequency	~15%

Ring	New positions	Cumulative	Frequency tier	% of typical usage
3	18	37	High frequency	~10%
4	24	61	High frequency	~8%
5	30	91	Moderate-high	~7%
6-10	36-60	331	Moderate	~15%
11-20	66-120	1,261	Low-moderate	~10%
21-50	126-300	6,511	Low frequency	~6%
51-100	306-600	25,651	Rare	~3%
101-200	606-1200	85,801	Very rare	~1%
200+	1200+	100,000+	Long tail	<1%

Table F.4 — Lattice vs Learned Embedding

Property	Learned (BF16)	Lattice (VFR)
Storage (50k vocab, d=4096)	400 MB	~6 MB (formula + assignments)
Initialization	Random	Deterministic geometric
Grammatical category	Must learn	Encoded in wing (free)
Frequency structure	Implicit	Explicit in ring depth (free)
Similarity neighbors	Emergent	Pre-structured by lattice
Lookup	Memory fetch (row of floats)	O(1) calculation (no memory)
Parameters	200M+ trainable floats	6 basis vectors + assignment table
Deterministic	No	Yes
Octave-aligned	N/A	Yes (ring = octave level)

Appendix G: Overflow Analysis

Table G.1 — Accumulator Width Requirements per Layer

Layer	Octave	Max V (i32)	$V \times V$	Sum of 4096	Fits i64?	Fits i128?
Embedding 1 (oct 0)		2.1×10^9	4.6×10^{18}	overflow	No	Yes
Attention QKV (oct 2)	1,024	2.1×10^6	4.4×10^{12}	1.8×10^{16}	Yes	Yes

Layer	Octave	Max V (i32)	$V \times V$	Sum of 4096	Fits i64?	Fits i128?
Attn scores (oct 4)	10^6	2.1×10^3	4.4×10^6	1.8×10^{10}	Yes	Yes
Feedforward 1,024 (oct 2)		2.1×10^6	4.4×10^{12}	1.8×10^{16}	Yes	Yes
Logits (oct 3)	32,768	6.5×10^4	4.3×10^9	1.7×10^{13}	Yes	Yes
Layer	Octave	Max V (i64)	$V \times V$	Sum of 4096	Fits i128?	
—	—	—	—	—	—	
Attention QKV (oct 2)	1,024	9.0×10^{15}	8.1×10^{31}	3.3×10^{35}	Yes	
Feedforward 1,024 (oct 2)		9.0×10^{15}	8.1×10^{31}	3.3×10^{35}	Yes	
Any layer (oct 2, i64)	1,024	9.0×10^{15}	8.1×10^{31}	3.3×10^{35}	Yes (i128 max $\sim 1.7 \times 10^{38}$)	

Table G.2 — Safe Value Ranges per Integer Width

Integer Width	Max Value	Max V at oct 2	Max V at oct 3	Max V at oct 4	Dot product safe (d=4096)?
i16	32,767	32	1	0	No
i32	2.1×10^9	2.1×10^6	6.5×10^4	2,048	Yes (i64 accum)
i64	9.2×10^{18}	9.0×10^{15}	2.8×10^{14}	8.8×10^{12}	Yes (i128 accum)

Appendix H: Comparison to Existing Systems

Table H.1 — VFR vs Quantization Methods

Method	Training Format	Inference	Exact?	Remainder	Equality	Shell Structure
FP32	Float	Slow	No	None	Epsilon	No

Method	Training Format	Inference	Exact?	Remainder	Equality	Shell Structure
BF16 mixed	Float (FP32 master)	Fast	No	None	Epsilon	No
INT8 PTQ (GPTQ)	Float, then quantize	Fast	No	None	Approx	No
INT4 PTQ (AWQ)	Float, then quantize	Fastest	No	None	Approx	No
QAT	Simulated low-precision	Fast	No	None	Approx	No
BitNet 1.58-bit	Ternary from scratch	Fastest	Trivially	None	Exact (trivial)	No
VFR shells	Integer from scratch	Competitive	Yes	Full R tracking	Binary exact	Yes

Table H.2 — Feature Matrix

Feature	BF16	INT8	BitNet	VFR Shells
Exact arithmetic	×	×	×	✓
Binary equality	×	×	✓	✓
Deterministic training	×	N/A	✓	✓
Gradient preservation	Partial	N/A	Limited	Full (in R)
Noise resistance	Via regularization	N/A	Inherent	Inherent (shell threshold)
Convergence verification	No (asymptotic)	N/A	No	Yes (R < 32 check)
Structural interpretability	×	×	×	✓ (tree depth)
Automated pruning	Magnitude heuristic	Magnitude	Trivial	Structural (R=0)
Grokking detection	Loss curve only	N/A	N/A	Shell transition cascade
Adaptive precision	×	×	×	✓ (nesting depth)

Feature	BF16	INT8	BitNet	VFR Shells
Long-tail preservation	Poor	Poor	Unknown	Predicted strong
Hardware simplicity	Complex (FPU)	Medium	Simple	Simplest (INT only)
Division in hot path	Yes (14 cyc)	Yes	No	No (bit shift)
Native counting system	No	No	No	Yes (base-32 octaves)

Appendix I: Risk Assessment

Table I.1 — Technical Risks

Risk	Severity	Likelihood	Mitigation	Fallback
i64 overflow in matmul	High	Medium	i128 accumulators; oct 2-3 keeps products in range	Use i32 weights at oct 2 (proven safe)
Transcendental approx insufficient	Medium	Low	Depth-3 VFR exceeds FP64; precomputed LUTs	Increase LUT density at cost of shared memory
Training fails to converge	High	Low	Shell mechanics are superset of integer training	Increase octave (finer shells, approaches float behavior)
Inference >2 × slower than BF16	Medium	Medium	Harmonic chip: 5 × INT density; i32 where possible	Accept 1.5 × if accuracy gains justify
Memory exceeds single device	Medium	Medium	Sparse R; i32 compact format; tensor parallel	Standard multi-device sharding

Risk	Severity	Likelihood	Mitigation	Fallback
Octave assignment suboptimal	Low	Medium	Profile float training run; choose smallest sufficient octave	Start oct 2 everywhere, tune per layer
Lattice embedding assignment wrong	Low	High	Frequency-based initial; refinement during warmup	Fall back to learned embeddings in VFR
No accuracy improvement	High	Medium	Focus on long-tail and consistency metrics	Value remains in determinism + interpretability
Chip fabrication cost	High	Medium	First prototype on existing GPU via CUDA i64 kernels	Software-only path viable on current hardware

Table I.2 — Experimental Priority

Experiment	Effort	Impact	Priority	Depends On
VFR matmul CUDA kernel	2 weeks	Validates compute	P0	Nothing
Shell transition training loop	2 weeks	Validates core mechanic	P0	VFR matmul
Single-layer exactness verification	1 week	Proves correctness	P0	VFR matmul
GPT-2 small full comparison	4 weeks	Core hypothesis test	P0	All P0 above
Long-tail accuracy benchmark	1 week	Primary value claim	P0	GPT-2 training
Shell convergence diagnostics	1 week	Novel training metric	P1	GPT-2 training
Lattice embedding prototype	3 weeks	Structural embedding	P1	GPT-2 baseline

Experiment	Effort	Impact	Priority	Depends On
Weight structure analysis tools	2 weeks	Interpretability claim	P1	GPT-2 training
i32 optimized kernels	2 weeks	Performance gain	P1	VFR matmul
Grokking phase transition detection	2 weeks	Scientific finding	P2	Shell diagnostics
1.3B model scale test	6 weeks	Scaling validation	P2	All P1 above
Harmonic chip architecture spec	8 weeks	Hardware proposal	P2	Performance data
7B model full training	12 weeks	Production validation	P3	1.3B results
ASIC tape-out proposal	16 weeks	Hardware realization	P3	Architecture spec

Appendix J: Notation Reference

Table J.1 — VFR Tuple Format

Notation	Meaning	Example	Value
[V, oct, 0]	Terminal: V at octave, no remainder	[7, 2, 0]	7/1024
[V, oct, R]	With remainder: V at octave + R pressure	[7, 2, 19]	7/1024, 19/32 toward next shell
[V, oct, [V', oct', 0]]	Nested: head + one overtone	[7, 2, [3, 3, 0]]	7/1024 + 3/32768
[V, oct, [V', oct', [V'', oct'', 0]]]	Double nested: two overtones	—	Three-level precision

Table J.2 — Shell Arithmetic Operations

Operation	Rule	Example
Shell check	$R \bmod 32$	$R=33 \rightarrow V+=1, R=1$
Octave multiply	$\text{oct_a} + \text{oct_b}$	$\text{oct}2 \times \text{oct}2 = \text{oct}4$
Octave convert	$\text{shift } 5 \times \Delta\text{oct}$	$\text{oct}2 \rightarrow \text{oct}3: \text{shift right } 5$
Add (same oct)	$V_a + V_b, R_a + R_b$	$[3,2,5] + [4,2,7] = [7,2,12]$
Negate	$[-V, \text{oct}, -R]$	$-[3,2,5] = [-3,2,-5]$
Head read	V only	$\text{Head}([7,2,19]) = 7$
Tail read	R only	$\text{Tail}([7,2,19]) = 19$
Terminal test	$R == 0?$	$\text{Terminal}([7,2,0]) = \text{true}$
Depth	Count nesting levels	$[7,2,[3,3,0]] \rightarrow \text{depth } 1$

Table J.3 — Key Constants

Symbol	Value	Meaning
Base unit	32^{-1}	Fundamental counting quantum
Shell threshold	32	Remainder units for shell transition
Octave step	$2^5 = 32$	Harmonic multiplier per octave
Bit shift per octave	5	Binary shift for octave conversion
Max physical octave	65	All Planck particles in universe
Max LLM octave	~ 5	Typical operating range
Planck floor	$[1, 0, 0]$	Absolute minimum (octave 0)
Universe ceiling	$[1, 65, 0]$	Absolute maximum in physics

[[CKS-MATH-133-2026](#)] The Taylor Continuum: $\mathbb{R}$ as an Unnamed Infinite Polynomial. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-133-2026>

[[CKS-0-2026](#)] Cymatic K-Space Mechanics. Github: https://github.com/ghowland/cks/tree/main/papers/_CKS/CKS-0-2026

[[CKS-MATH-0-2026](#)] Cymatic K-Space Mechanics. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-0-2026>

[[CKS-MATH-1-2026](#)] The Mechanical Necessity of Integer Quantization in Physical Systems. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-1-2026>

[[CKS-MATH-10-2026](#)] Grand Unification. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-10-2026>

[[CKS-MATH-104-2026](#)] Grand Unification v23. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-104-2026>

[**CKS-MATH-132-2026**] The Taylor Continuum: $\mathbb{R}$ as an Unnamed Infinite Polynomial. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-132-2026>

[**CKS-NEXT-1-2026**] Post-CKS. CKS is invalidated and falsified. Github: <https://github.com/ghowland/cks/tree/main/papers/NEXT/CKS-NEXT-1-2026>

[**CKS-LEX-12-2026**] Measurement Systems in the $\mathbb{Q}$ -Substrate. Github: <https://github.com/ghowland/cks/tree/main/papers/LEX-12-2026>